

0.0.1 Sobre escrita *Override*

A sobre escrita de métodos, ou *override*, é um conceito central na programação orientada a objetos que permite que sobrescrever um método de outra classe. Isso será tratado com mais detalhes no próximo capítulo. Em Java a sobrescrita é feita utilizando a anotação `@Override` antes da declaração do método. Essa anotação não é obrigatória, mas é altamente recomendada, pois ajuda o compilador a verificar se você está realmente sobrescrevendo um método existente na superclasse. Se houver algum erro, como uma assinatura incorreta, o compilador emitirá um aviso.

0.0.2 Método `toString`

O método `toString()` é um componente fundamental da classe `Object` em Java. Por padrão, quando tentamos exibir um objeto diretamente, o Java retorna um código hexadecimal pouco compreensível (como `Pet@15db9742`). Portanto, o método `toString()` é extremamente útil para exibir as informações de um objeto de forma legível para humanos. Para que essa representação seja útil no nosso sistema de pets, podemos sobrescrever esse método em nossas classes personalizadas, garantindo que ele retorne os dados que realmente importam.

```
public class Pet {
    private String nomeAnimal;
    private int idadeAnimal;
    private int sexoAnimal;

    public Pet(String nome, int idade, int sexo) {
        this.nomeAnimal = nome;
        this.idadeAnimal = idade;
        this.sexoAnimal = sexo;
    }
    // Sobrescrita do metodo, abordaremos no proximo capitulo
    @Override
    public String toString() {
        return "Pet [Nome: " + nomeAnimal + ", Idade: " +
            idadeAnimal + " anos, " + Sexo: " + sexoAnimal]";
    }
}

// No uso do sistema:
Pet meuPet = new Pet("Rex", 3);
System.out.println(meuPet);
// Saída no console: Pet [Nome: Rex, Idade: 3 anos, Sexo: 1]
```

0.0.3 Método `equals`

Como descrito no início do capítulo, a identidade do objeto é única e imutável, portanto, é muito difícil de comparar dois objetos, mesmo que eles possuam os mesmos atributos

e valores. Para isso, o Java fornece o método `equals()` que compara se dois objetos são iguais, ou seja, se possuem os mesmos valores de atributos.

Para utilizar esse método, precisamos, sobrescrever o método `equals()` na classe que desejamos comparar. Abaixo segue um exemplo de como implementar o método `equals()` na classe `Pet`.

```
public class Pet {
    private String nomeAnimal;
    private int idadeAnimal;
    private int sexoAnimal;
    private boolean vacinado;

    // Construtor e outros metodos omitidos

    @Override
    public boolean equals(Object obj) {
        if (obj instanceof Pet) {
            Pet p = (Pet) obj;
            return this.nomeAnimal.equals(p.nomeAnimal) &&
                   this.idadeAnimal == p.idadeAnimal &&
                   this.sexoAnimal == p.sexoAnimal;
        }
        return false;
    }
}
```

Observação

O operador `instanceof` em Java é uma palavra-chave utilizada para realizar testes de tipo em tempo de execução (*runtime type checking*). Ele permite verificar se um objeto é uma instância de uma classe específica, se implementa uma determinada interface ou se pertence a uma subclasse.

0.0.4 Atributos estáticos

Até agora nesse capítulo utilizamos métodos e atributos de instância, ou seja, cada objeto possui seus próprios valores. No entanto, existem situações em que queremos que um atributo ou método seja compartilhado entre todos os objetos de uma classe. Para isso, utilizamos o modificador `static`. Atributos e métodos estáticos pertencem à classe em si, e não a instâncias individuais. Isso significa que eles podem ser acessados diretamente através do nome da classe, sem a necessidade de criar um objeto.

No exemplo da clinica, não faz muito sentido o uso de atributos estáticos, mas um exemplo seria manter o números de pets cadastrados.

```
...
    private static int contadorPets = 0;
...
```

Observação

Por isso, eles só podem acessar outros membros estáticos da mesma classe. Essa restrição existe porque, ao invocar um método estático, não há uma instância associada — ou seja, não temos acesso à referência *this*. Como o *this* representa o objeto em execução e métodos estáticos são chamados diretamente via nome da classe, torna-se impossível acessar dados que dependeriam de um objeto particular para existir (Cabral (2014)).

Um novo exemplo de uso de atributos estáticos é a classe `Math` do Java, que contém métodos e constantes matemáticas. Por exemplo, a constante `Math.PI` representa o valor de pi e é acessada diretamente pela classe, sem a necessidade de criar uma instância da classe `Math`. Veja o exemplo abaixo:

```
public class ExemploMath {
    public static void main(String[] args) {
        double raio = 5.0;
        double area = Math.PI * Math.pow(raio, 2); // nao e
            necessario criar um objeto da classe Math para acessar
            a constante PI e o metodo pow()
        System.out.println("Area do circulo: " + area);
    }
}
```

0.1 Representação de classes na UML (Linguagem de Modelagem Unificada)

A UML é uma linguagem visual padronizada para documentar, especificar e projetar sistemas. Essencial em engenharia de software, ela transforma lógicas e códigos complexos em diagramas de fácil compreensão, divididos entre visões estruturais e comportamentais. Dentro da UML existe o diagrama de classes.

Uma classe é representada por um retângulo dividido em Nome, Atributos e Métodos. Mais tarde serão adicionados sinais que definem o tipo de encapsulamento dos atributos e métodos. A Figura 1 representa a classe `Pet`.

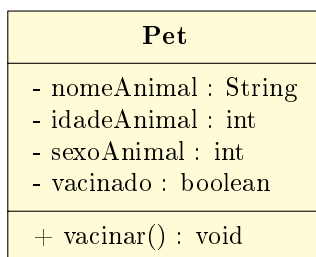


Figura 1: Diagrama UML da classe `Pet`.

A UML não é foco desta apostila, mas é interessante representar as classes de forma padronizada. Ao longo dos próximos capítulos e seções serão inseridos novos tipos de representações.

0.2 Resumo dos principais tópicos abordados

A tabela 1 abaixo resume os principais conceitos abordados neste capítulo, fornecendo uma visão geral das definições e exemplos discutidos.

Tabela 1: Resumo dos conceitos fundamentais de POO

Termo	Breve Explicação
Abstração	Generalização das características essenciais de um objeto para resolver um problema.
Objeto	Representação concreta de uma entidade; possui estado (atributos) e comportamento (métodos).
Classe	O "molde" ou modelo que define os atributos e comportamentos dos objetos.
Instância	O objeto concretizado em memória, criado a partir de uma classe.
Encapsulamento	Técnica de ocultar detalhes internos, expondo apenas uma interface pública e segura.
Atributos	As características ou dados que definem o estado de um objeto.
Métodos	As ações ou comportamentos que um objeto pode realizar.
Construtor	Método especial chamado ao instanciar um objeto para inicializar seu estado.
Sobrecarga	Capacidade de ter múltiplos métodos com o mesmo nome, mas assinaturas (parâmetros) diferentes.
Static	Modificador que faz atributos/métodos pertencerem à classe, não a uma instância específica.

1 Relacionamentos entre classes

Uma das maiores vantagens da Orientação a Objetos é o reuso de código, que permite aproveitar implementações existentes para criar novas funcionalidades. Além disso, a POO facilita a modelagem de sistemas complexos através do relacionamento entre objetos, permitindo que eles colaborem entre si para executar tarefas maiores.

1.1 Herança

Considere a necessidade de cadastrar os proprietários dos pets em nosso sistema, utilizando os atributos **nome**, **cpf** e **e-mail**. Paralelamente, precisamos gerenciar os profissionais da clínica, como os veterinários, que compartilham os mesmos dados básicos de identificação, mas possuem atributos adicionais, como o **CRMV** (Conselho Regional de Medicina Veterinária).

Fica o questionamento, precisamos criar duas classes praticamente idênticas só pela diferença do atributo CRMV? E se existir a necessidade de adicionar um novo atributo comum à cliente e veterinário, teríamos que percorrer todas classes para realizar essa mudança?

Com a herança, é possível economizar linhas de código e consequentemente tempo desenvolvendo programas baseando novas classes existentes em classes existente. Ao criar uma classe, em vez de declarar membros novos, você pode designar que a nova classe, chamada de **subclasse**, deva **herdar** membros de uma classe existente, que é chamada de **superclasse** (Deitel and Deitel (2015)).

Observação

Podemos dizer que um objeto **é um tipo de** outro objeto, quando precisamos utilizar herança.

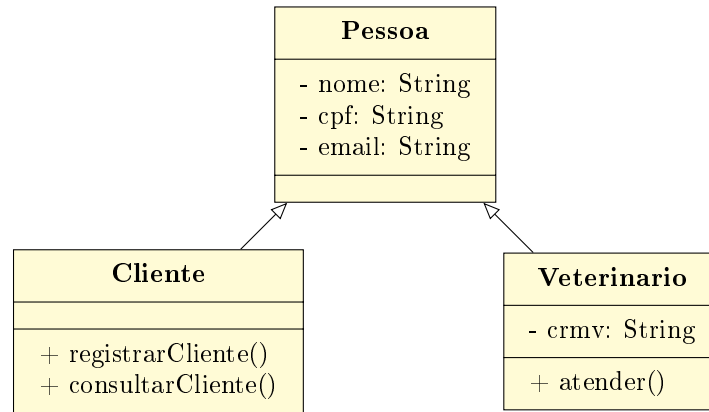
1.1.1 Tipos de herança

Na POO Herança possui dois tipos:

- **Simples:** Ocorre quando uma subclasse possui apenas uma superclasse. O objetivo é especializar e reutilizar membros de um único conceito de classe base. Veja o exemplo abaixo
- **Herança Múltipla:** Ocorre quando uma subclasse necessita de duas ou mais superclasses, permitindo especializar mais de um conceito da aplicação simultaneamente. Novos exemplos serão apresentados nos próximos capítulos.

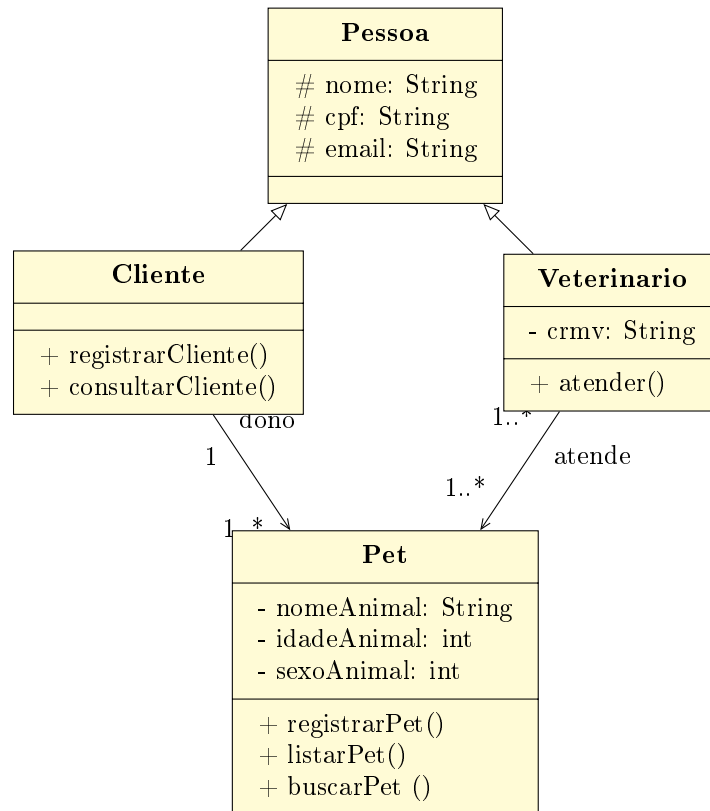
1.1.2 Herança simples

Antes de iniciarmos a implementação, vamos entender como a herança funciona na UML. Uma herança, na UML, é representada por seta entre uma subclasse e uma superclasse. Para representar os modificadores de acesso (??) usamos – para private, + para public e # para protect.



Em herança simples, uma subclasse tem a capacidade de estender a superclasse, incorporando seus próprios atributos e métodos exclusivos. Por essa razão, ela é considerada mais específica e representa uma **especialização** da classe original. Além de herdar os comportamentos da superclasse, a subclasse pode **sobrescrevê-los** para adaptá-los às suas necessidades particulares, reforçando o conceito de que a herança funciona como um mecanismo de **especialização** (Deitel and Deitel (2015)). No exemplo UML abaixo vamos integrar nossa classe Pet ao diagrama. Retiramos o atributo vacinado, pois utilizamos apenas de forma didática anteriormente.

1 Relacionamentos entre classes



Para acessar os atributos herdados sem comprometer o encapsulamento utilizamos o modificador de acesso **protected**, que se posiciona entre o **private** e o **public** em termos de visibilidade. Um atributo ou método **protected** é acessível pela própria classe e por suas subclasses, garantindo que o acesso seja restrito à hierarquia de herança, mantendo a integridade dos dados protegida contra acessos externos não autorizados.

Observação

Em Java, para uma classe herdar outra classe utilizamos a palavra **extends**, após o nome da classe seguido do nome da superclasse.

Veja o exemplo abaixo, sobre a herança simples em Java.

```
// Superclasse
class Pessoa {
    // Usamos protected para permitir acesso pelas subclasses,
    // mas esconder do restante do sistema.
    protected String nome;
    protected String cpf;
    protected String email;

    public Pessoa(String nome, String cpf, String email) {
        this.nome = nome;
    }
}
```

```

        this.cpf = cpf;
        this.email = email;
    }
}

// Subclasse
class Veterinario extends Pessoa {
    private String crmv;

    public Veterinario(String nome, String cpf, String crmv) {
        super(nome, cpf, email); // Chama o construtor da
                                // superclasse
        this.crmv = crmv;
    }

    public void exibirInformacoes() {
        // Acesso direto aos atributos protected herdados
        System.out.println("Nome: " + this.nome);
        System.out.println("CPF: " + this.cpf);
        System.out.println("CRMV: " + this.crmv);
    }
}

```

Além do exemplo apresentar o uso do **extends**, a palavra **super** é usada para chamar o construtor da superclasse. Ainda no exemplo, pode-se observar o acesso aos atributos da classe “mãe”, pois seu acesso é do tipo *protected*. Para o acesso, basta usar a palavra **this**.

Observação

Por uma questão de organização, é boa prática criar um arquivo .java para cada classe.

1.1.3 Herança múltiplas

A herança múltipla ocorre quando uma subclasse necessita não de apenas uma, mas duas ou mais classes para herdar. Leite et al. (2016). Dessa forma a classe filha herda os atributos e métodos de todas as classes que a compõem. A herança múltipla é um recurso muito avançado e deve ser usado com cautela, pois pode introduzir complexidade e ambiguidade no design do sistema.

O exemplo abaixo de uma representação de herança múltipla na UML, onde a classe **CaoGuarda** herda de duas superclasses, **Pet** e **AnimalTrabalho**. A classe **CaoGuarda** possui atributos e métodos próprios, além de herdar os atributos e métodos das superclasses. A seta indica a relação de herança entre as classes, mostrando que **CaoGuarda** é uma especialização de ambas as superclasses.

1 Relacionamentos entre classes

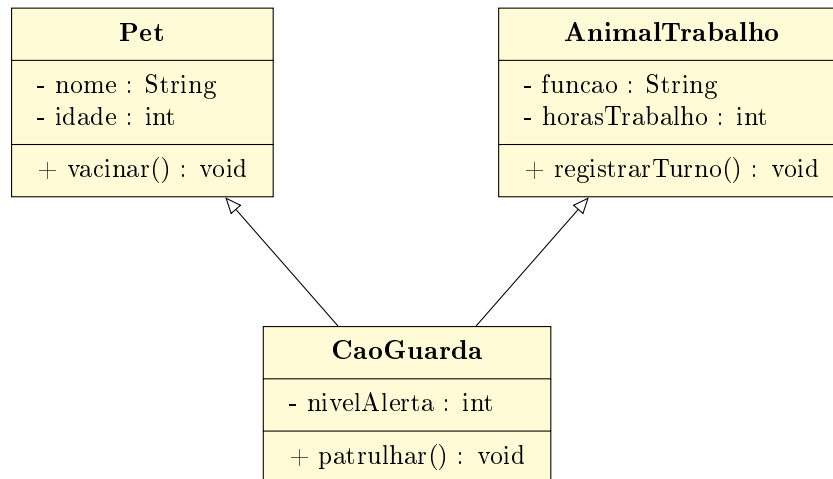


Figura 1.1: Exemplo conceitual de herança múltipla em UML entre classes.

Observação

Em Java, a herança múltipla de classes não é suportada diretamente, mas pode ser alcançada através do uso de interfaces, assunto esse que veremos no próximo capítulo.